

Load Balancing Algorithms

Part 2

Prepared By: Dr. D. P. Singh

IP-Hash

- IP hash load balancing is a technique that **directs client requests to specific servers based on a hash of their source (and sometimes destination) IP address.**
- It ensures consistent session persistence, as the **same client IP is mapped to the same backend server**, making it ideal for applications requiring session memory, such as shopping carts.

IP-Hash

contd...

Advantages:

- **Session Persistence (Sticky Sessions):** If a client is disconnected, their subsequent request is **directed to the same server** because the hash key is regenerated.
- **Consistency:** The algorithm guarantees that a client (from the same IP) is consistently routed to the same server, provided the server list remains unchanged.
- Primarily used in **e-commerce, banking, or any web application where user data must be retained on a specific server throughout a session.**

IP-Hash

contd...

Disadvantages:

- If a server becomes unavailable, the mapping changes, breaking the session.
- It may also lead to uneven load distribution if many users are behind a single proxy IP (e.g., in a corporate office).

Weighted Algorithms

- Weighted load balancing algorithms distribute traffic **based on pre-assigned capacity ratios**, directing more traffic to stronger servers.
- Key types include **Weighted Round Robin** (sequential assignment based on weight) and **Weighted Least Connections** (assigning to the least loaded server among the highest-weighted).
- These improve performance for heterogeneous server setups by optimizing resource usage.

Weighted Algorithms

contd...

- **Weighted Round Robin:** Assigns incoming requests in a circular manner, **but servers with higher weights (representing higher capacity) receive more traffic**. For example, a 3:2:1 ratio sends 3 requests to Server A, 2 to Server B, and 1 to Server C.
- **Weighted Least Connections:** Selects the server with the lowest ratio of active connections to weight. **This is ideal for scenarios with varying request processing times.**

Weighted Algorithms

contd...

Advantages:

- **Heterogeneous Environments:** Ideal **when backend servers** have different processing powers, RAM, or storage capabilities.
- **Traffic Management:** Allows administrators to direct more traffic to **newer or more powerful machines**.
- **Efficiency:** Prevents under-utilization of powerful servers while **protecting weaker ones from overload**.

Weighted Algorithms

contd...

Disadvantages:

- **Static Configuration Issues:** **Weights are typically assigned manually**, meaning the system cannot automatically adjust to unexpected server performance degradation or traffic surges.
- **Lack of Real-Time Adaptation:** They do not account for current server load, CPU usage, or memory availability, **relying instead on pre-set, static capacity assumptions**.
- **Complexity:** Compared to simple Round Robin, maintaining and updating weighted tables requires more administrative overhead, which can lead to misconfiguration.
- **Inefficient for Varying Requests:** In algorithms like Weighted Round Robin, if different requests take significantly different amounts of time, the load can still become imbalanced, **causing stronger servers to be underutilized**.

Geographical Algorithms

- Geographical load balancing (or Global Server Load Balancing - GSLB) **distributes user traffic across servers in multiple, physically dispersed data centers**, typically using DNS to route requests to the nearest or healthiest node.
- This reduces latency, improves reliability via failover, and helps meet data compliance requirements.
- **How it Works (GSLB):** When a user requests access, a GSLB-enabled DNS server analyzes the user's location and the real-time health/load of available servers to direct traffic to the optimal location.
- **Performance & Latency:** By connecting users to the nearest server, data travels a shorter distance, **resulting in faster load times and better user experience**.
- **Reliability & Disaster Recovery:** If a specific datacenter goes down, traffic is automatically rerouted to the next closest, functional location, preventing downtime.

Geographical Algorithms

contd...

Advantages:

- **Improved User Experience:** Reduced latency.
- **High Availability:** Automatic failover.
- **Compliance:** Routing data based on regional regulations.
- **Cost Efficiency:** Using cheaper, renewable, or idle resources.

Geographical Algorithms

contd...

Disadvantages:

- **High Complexity and Cost:** Implementing and managing infrastructure across multiple geographic regions is complex, increasing operational, hardware, and software costs.
- **Data Synchronization Issues:** Maintaining data consistency across geographically distributed databases is challenging and can introduce latency.
- **DNS Propagation Delays:** Changes in traffic routing may take time to propagate, potentially causing delays in adjusting to load changes.

Consistent Hashing

- Consistent hashing is a distributed load-balancing technique that maps servers and requests onto a virtual "hash ring". By **assigning requests to the first server found moving clockwise on the ring.**
- **Map Servers:** Hash server IDs to positions on the ring.
- **Map Requests:** Hash incoming request keys to positions on the same ring.
- **Assign Request:** Move clockwise from the request's position; the first server encountered handles the request.
- **Handle Churn:** If a server is added/removed, only requests in the immediate, affected segment of the ring are remapped.

Working of Consistent Hashing

Distributes keys across nodes while minimizing rehashing when nodes change.

- ✓ Scalable
- ✓ Fault Tolerant
- ✓ Minimal Data Movement

1 Hash everything onto a circular ring

Each key is hashed → Hash() → Position on ring

Each server (node) is hashed → Hash() → Position on ring

Hashes are placed on a circle ($0 \rightarrow 2^{32}-1$) for uniform distribution

Legend:

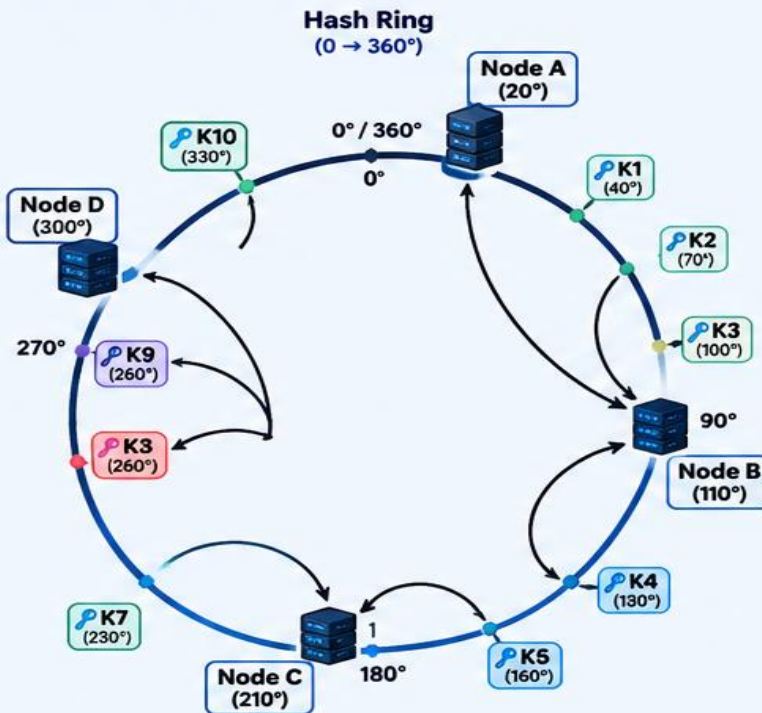
 = Server (Node)

 = Data Key

 = Key mapped to nearest server (clockwise)

 = Keys that move after node is added

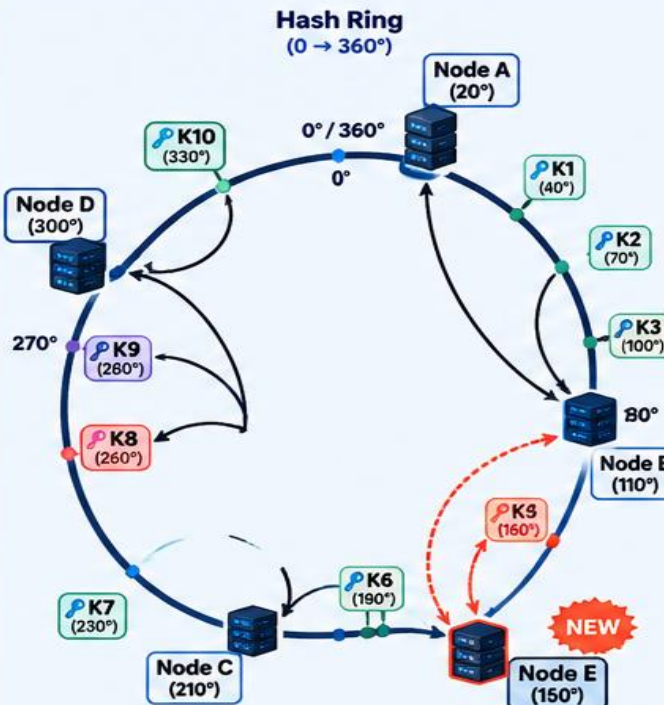
2 Keys map to the next server clockwise



Key → Server Mapping

K1, K2, K3	→ Node B	K8, K9, K10	→ Node D
K4, K5	→ Node C		
K6, K7	→ Node C		

3 What happens when a new node is added?



After adding Node E:

- ✓ Only keys in the range (110° – 210°) are remapped to the new node.
- ✓ K4, K5 now go to Node E
- ✓ All other keys stay on the same servers!
- ✓ **Minimal data movement** ✓

4 Why is it powerful?



Easy to scale

Add / remove servers with minimal rebalancing



Fault tolerant

When a node fails, only its keys move to the next server



Even distribution

Hashing gives balanced load across servers



Perfect for distributed systems

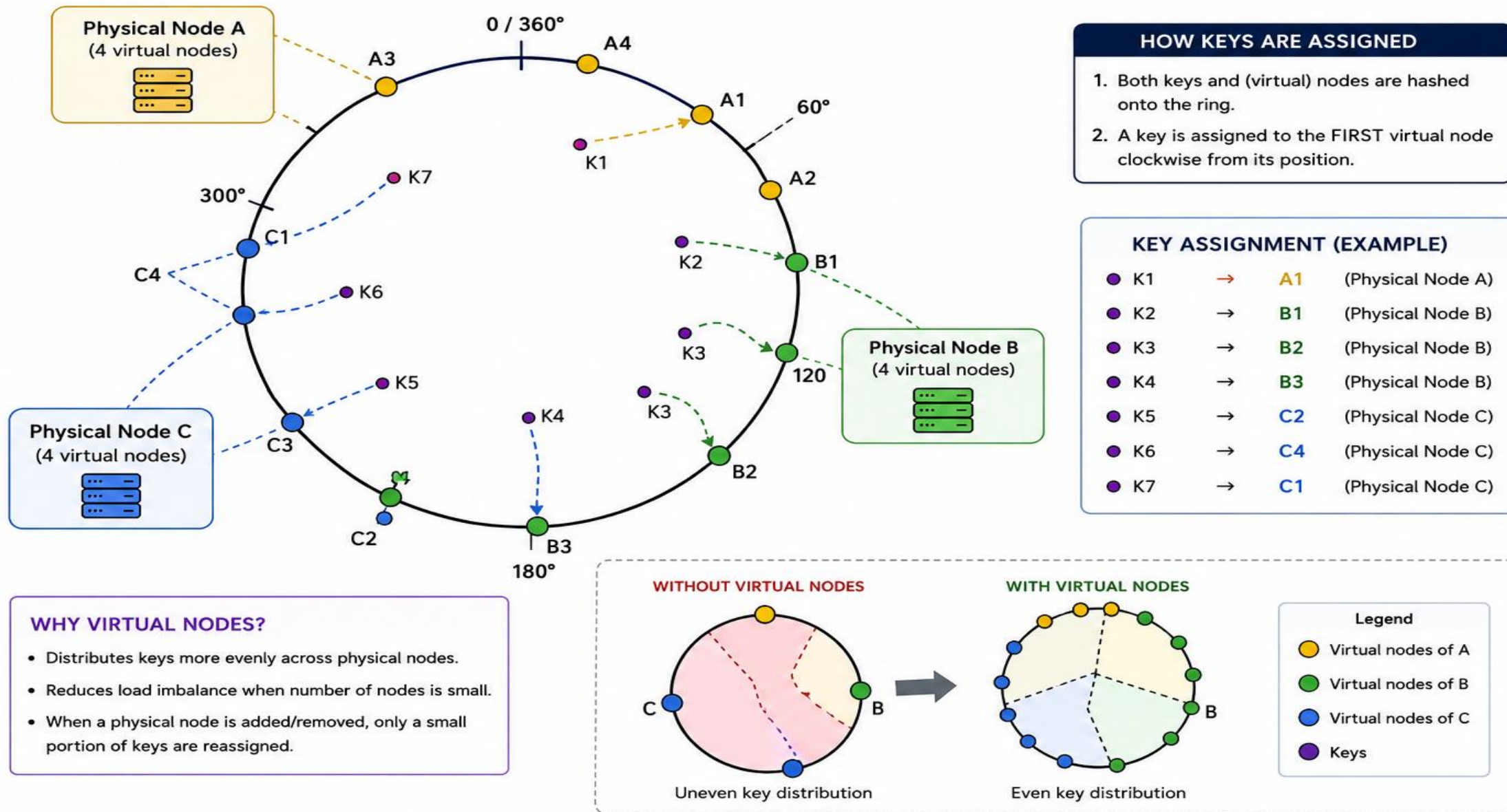
- ✓ Used in:
- Caches (Redis, Memcached)
 - Databases (Cassandra)
 - CDNs
 - Load Balancers



In short: **Consistent Hashing** keeps the system balanced, scalable and efficient by remapping only a small portion of keys when the cluster changes.

VIRTUAL NODES IN CONSISTENT HASHING

Each physical node is represented by many virtual nodes on the hash ring.



- Thank You